Morpho: Midnight - BlueBuyCallback [93db7e08] Security Review

Auditors

MiloTruck, Lead Security Researcher

Saw-mon and Natalie, Lead Security Researcher

StErMi, Lead Security Researcher

Report prepared by: Lucas Goiriz

August 3, 2026

Contents

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

Morpho is a trustless and efficient lending primitive with permissionless market creation.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of Morpho: Midnight - BlueBuyCallback [93db7e08] according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

Over the course of 0 days in total, Morpho engaged with Spearbit to review the midnight protocol. In this period of time a total of **15** issues were found.

Summary

Project Name	Morpho
Repository	midnight
Commit	93db7e08
Type of Project	DeFi, Lending
Audit Timeline	Jul 24th to Jul 24th

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	3	1	2
Gas Optimizations	0	0	0
Informational	12	7	5
Total	**15**	**8**	**7**

The Spearbit team reviewed Morpho's midnight holistically on commit hash eba62f70 and concluded that all findings were addressed and no new vulnerabilities were identified.

4.1 Scope

The security review had the following components in scope for midnight on commit hash 93db7e08:

```
src/periphery
├── BlueBuyCallback.sol
└── BlueBuyCallbackFactory.sol
```

5 Findings

5.1 Low Risk

5.1.1 `buyerAssetsBound` reports unusable liquidity for entry-gated buyers

Severity: Low Risk.

Context: BlueBuyCallback.sol#L87-L90.

Description: Let $B_{\text{Blue}} > 0$ be the assets withdrawable by `BlueBuyCallback`. The function returns B_{Blue} while ignoring `id`, `market`, and `buyer`. It therefore also reports a positive bound for `buyer != OWNER`, although `onBuy` always rejects that buyer.

Assume the buyer has no debt and the market entry gate rejects or reverts for `canIncreaseCredit(buyer)`. For every $U > 0$:

$$\texttt{buyerCreditIncrease} = \max(U - \texttt{buyer.debt}, 0) = U > 0$$

Consequently, `Midnight.take` reverts for every positive-unit purchase, but `buyerAssetsBound` still returns $B_{\text{Blue}} > 0$. A router using the advertised bound can therefore select an unexecutable take.

The current API also cannot return the exact gate-aware bound when the buyer has debt. Such a buyer may purchase up to its debt without entering, but converting that unit limit into `buyerAssets` requires the offer direction and price. For the same reason, the endpoint cannot account for remaining asset/unit caps or maker-side `reduceOnly`.

Recommendation: At minimum, return.

- Zero when `buyer != OWNER`, or.
- When the buyer has zero debt and `market.enterGate` rejects or reverts for `canIncreaseCredit(buyer)`.

For an exact executable bound, change the API to accept the `Offer`. Bound the result by Blue liquidity, remaining offer capacity, and the assets corresponding to the buyer's debt whenever entry is denied or a maker-buy offer is `reduceOnly`.

> ℹ️ **Note:**
> In general the exact purpose for this queryable function is not clear since the specifications are not provided and most of the input parameters are ignored.

Footnote: `buyerAssetsBound` does not also consider inconsistencies between IRM accounting when:

- `borrowRate`
- `borrowRateView`

Morpho: Acknowledged. The expected behavior and limitations of `buyerAssetsBound` were documented in PR 1088.

Spearbit: Acknowledged.

5.1.2 `buyerAssetsBound` should return zero for non loan tokens

Severity: Low Risk.

Context: BlueBuyCallback.sol#L90.

Description/Recommendation: If the Midnight market loan L_M token does not match the decoded Blue market's loan token L_B, `buyerAssetsBound` should return `0`.

Morpho: Acknowledged. The `wrong loan token` case is documented in PR 1088.

Spearbit: Acknowledged.

5.1.3 `buyerAssetsBound` reverts when the provided `data` is short

Severity: Low Risk.

Context: BlueBuyCallback.sol#L91.

Description: If `data` does not occupy enough memory so, decoding it into `MarketParams` can revert. An example test would be:

```
  diff --git a/test/BlueBuyCallbackTest.sol b/test/BlueBuyCallbackTest.sol
  index ab34dbb1..9fb19fcd 100644
-  -- a/test/BlueBuyCallbackTest.sol
+  ++ b/test/BlueBuyCallbackTest.sol
  @@ -204,6 +204,11 @@ contract BlueBuyCallbackTest is Test {
           assertEq(result, 0);
       }

+      function testBuyerAssetsBoundRevertsWhenDataIsSmall() public {
+          uint256 result = callback.buyerAssetsBound(bytes32(0), market, owner, hex"");
+          assertEq(result, 0);
+      }
+
       function testOnBuyRevertsIfCallerIsNotMidnight(address caller) public {
           vm.assume(caller != address(midnight));
           vm.expectRevert(IBlueBuyCallback.NotMidnight.selector);
```

Recommendation: If in `buyerAssetsBound` the length of `data` is less than the memory required to decode `MarketParams` (5 words) return `0`. One could also explicitly revert depending on the design decision.

Morpho: Further clarification has been added to the NatSpec:

```
/// @dev Reverts if data is not well formed.
```

on PR 1088.

Spearbit: Fix verified.

5.2 Informational

5.2.1 `BlueBuyCallback` lacks a skim endpoint for stranded assets

Severity: Informational.

Context: BlueBuyCallback.sol#L23.

Description: Let B_N be the native-token balance of `BlueBuyCallback`, and let B_T be its balance of an arbitrary ERC-20 token T.

Native tokens can be forced or prefunded to the callback address, and any ERC-20 can be transferred to it. However, the callback's only asset-related flow is `onBuy`, which withdraws the exact requested loan-token amount

from Blue and approves Midnight to pull it. Neither this flow nor the owner-controlled endpoints can transfer an existing balance to `OWNER`.

Consequently, accidental or forced balances satisfy.

$$B_N > 0 \ \vee \ B_T > 0 \quad \Longrightarrow \quad \text{no owner-controlled recovery path.}$$

These assets remain stranded in the callback.

Recommendation: Add an `OWNER`-only `skim` endpoint that transfers the callback's full balance of either the native token or an arbitrary ERC-20 to a specified receiver. Emit the token, receiver, and amount so the recovery is reconstructible from events. Use a safe ERC-20 transfer helper and bubble or explicitly handle native-transfer failure.

Morpho: Fixed in PR 1096.

Spearbit: PR 1096 adds a `skim` function that allows any caller to transfer stranded ERC-20 tokens held by the contract to `OWNER`, mirroring the design used in `MorphoMarketV1AdapterV2`.

5.2.2 `BlueBuyCallbackFactory.createBlueBuyCallback` should track the `msg.sender`

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: Anyone can call the `BlueBuyCallbackFactory.createBlueBuyCallback` function and deploy a `BlueBuyCallback` contract on behalf of an arbitrary `owner`.

The function should track in the `CreateBlueBuyCallback` event the `caller` of the function given that it could be different from the `owner` input parameter.

Recommendation: Morpho should add `address indexed caller` to the parameters of the `CreateBlueBuyCallback` event definition.

Morpho: Fixed in PR 1087.

Spearbit: Fix verified.

5.2.3 `BlueBuyCallbackFactory` should revert when incorrectly configured

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: The `BlueBuyCallbackFactory` does not perform any sanity checks in the constructor. If the `MIDNIGHT` and `BLUE` immutable variables are left uninitialized, any calls to the `createBlueBuyCallback` would generate ”useless” `BlueBuyCallback` contracts.

Recommendation: Morpho should consider adding basic sanity checks to the `BlueBuyCallbackFactory` constructor.

- `require(_midnight != address(0) && _midnight.code.length != 0);`
- `require(_blue != address(0) && _blue.code.length != 0);`

Morpho: Acknowledged. See Morpho Protocols Audit Guidelines.

Spearbit: Acknowledged.

5.2.4 `setAuthorization` and `setAuthorizationWithSig` should not allow the `OWNER` to remove self-unauthorize

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: Both the `setAuthorization` and `setAuthorizationWithSig` are allowing the `OWNER` to self-unauthorize itself on the `BLUE` contract as the authorized user to interact on Morpho Blue on behalf of the `BlueBuyCallback` contract.

The owner should **always** be authorized to be able to withdraw from the positions that are owned by the callback's contract (in addition to executing other actions that could require the authorization).

Recommendation: Morpho should:

- Revert `setAuthorization` when `authorized == OWNER && newIsAuthorized == FALSE`.
- Revert `setAuthorizationWithSig` when `authorization.authorized == OWNER && authorization.isAuthorized == FALSE`.

Morpho: Acknowledged. The `OWNER` can always call `setAuthorization(OWNER, true)` again to restore access to the Blue position if needed.

Spearbit: Acknowledged.

5.2.5 Consider using on demand exact allowance approval when `onBuy` is executed

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: The current implementation of `BlueBuyCallback.onBuy` is providing the `MIDNIGHT` contract an infinite allowance for the `market.loanToken` token even if the exact amount pulled by `MIDNIGHT` (after the callback return) is already known.

Recommendation: Morpho should consider replacing `forceApproveMax(market.loanToken, MIDNIGHT);` with `safeApprove(market.loanToken, MIDNIGHT, buyerAssets);`

Morpho: Fixed in PR 1094.

Spearbit: Fix verified.

5.2.6 Execute allowance approval only when tokens will be actually pulled by Midnight

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: When `buyerAssets` is zero, the `Midnight.take` function will pull an empty amount of tokens from the `BlueBuyCallback` contract (the `payer` in this case).

```
if (buyerCallback != address(0)) {
    bytes memory buyerCallbackData = offer.buy ? offer.callbackData : takerCallbackData;
    require(
        IBuyCallback(buyerCallback)
            .onBuy(id, offer.market, buyerAssets, units, buyerPendingFeeIncrease, buyer,
            ↪  buyerCallbackData)
        == CALLBACK_SUCCESS,
```

```
        WrongBuyCallbackReturnValue()
    );
}

SafeTransferLib.safeTransferFrom(offer.market.loanToken, payer, address(this), buyerAssets -
↪  sellerAssets);
SafeTransferLib.safeTransferFrom(offer.market.loanToken, payer, receiver, sellerAssets);
```

This means that no allowance will be consumed and there's no reason for the `BlueBuyCallback` contract to provide infinite allowance to the `Midnight` contract.

Recommendation: Morpho should consider moving `forceApproveMax(market.loanToken, MIDNIGHT);` inside the `if (buyerAssets > 0)` branch of the `BlueBuyCallback.onBuy` function.

Morpho: Acknowledged. Per Morpho's audit guidelines, no-op gas optimizations are not a concern; additionally, the approval is no longer infinite following PR 1094.

Spearbit: Acknowledged.

5.2.7 `buyerAssetsBound` should document if the callback contract can be configured as a BLUE fee recipient

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: If the `BlueBuyCallback` is configured as the fee recipient for the `BLUE` market, the value returned by `IMorpho(BLUE).position(marketParams.id(), address(this)).supplyShares` could not yet include the shares minted by `Morpho._accrueInterest`.

Recommendation: Morpho should document in the `buyerAssetsBound` natspec if the `BlueBuyCallback` can be configured as a `BLUE` fee recipient.

If that's the case the `buyerAssetsBound` should explicitly disclose that the internal logic could underestimate the `supplyAssets` value and so the final value returned.

Morpho: Fixed in PR 1085.

Spearbit: Fix verified.

5.2.8 `BlueBuyCallback` natspec should be further expanded

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: Other Morpho's contracts like the BlueBundlesV1 or Midnight have very extensive and detailed natspec documentation that covers which is the contract's scope, the explicit or implicit assumptions and implementation behaviors like no-op operations, zero checks, and so on.

Recommendation: Morpho should rewrite the `BlueBuyCallback` natspec to cover the following aspects.

- Contract's scope.
- Explicit and Implicit assumptions.
- Expected requirements.
- General implementations adopted standards like no-op operations, zero checks and so on.

- ”Custom” implementation behaviors relevant only to `BlueBuyCallback`.

Morpho: Fixed in PR 1099.

Spearbit: Fix verified.

5.2.9 Consider reducing the complexity of `BlueBuyCallback` flows derived by unrestricted delegation on Blue positions

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: The main goal of `BlueBuyCallback` is to be able to allow `OWNER` to park tokens in the `BLUE` markets to yield interest while waiting for possible `MIDNIGHT.take` actions (where the `OWNER` is the buyer).

The ”main” reason to set `OWNER` as an authorized user in `BLUE` for the `BlueBuyCallback` instance is to allow the `OWNER` to withdraw the callback's supply position (+ yield).

But because `BlueBuyCallback.setAuthorization` and `BlueBuyCallback.setAuthorizationWithSig` allow the `OWNER` to add/remove new authorized users on `MORPHO` for the `BlueBuyCallback` contracts it means that multiple users could manage the callback contract on `BLUE` which could include executing `borrow` flows (`supplyCollateral` can be executed by anyone on behalf of `BlueBuyCallback`).

Given the specific **core scope** of `BlueBuyCallback` Morpho should consider reducing what can be done with the contract's positions on Morpho Blue enabled by the authorization system.

1) Remove `IMorpho(BLUE).setAuthorization(OWNER, true);` from `constructor`.
2) Remove the `setAuthorization` function.
3) Remove the `setAuthorizationWithSig` function.
4) Add a `withdraw` function callable only by the `OWNER` that will call `BLUE.withdraw`.
5) Add a `withdrawCollateral` function callable only by the `OWNER` that will call `BLUE.withdrawCollateral`.
6) (optional) If there's a need for the `OWNER` to allow delegation, implement a ”local” delegation system to allow the owner to delegate the execution of `BlueBuyCallback` `withdraw` and `withdrawCollateral` functions.

Recommendation: Morpho should consider reducing the complexity of `BlueBuyCallback` flows derived by unrestricted delegation on Blue positions.

Morpho: Acknowledged. Indeed that would have been an other path. we are ok with the fact that someone could indeed borrow / deposit collateral etc through the callback, so we decided to ack the issue.

Spearbit: Acknowledged.

5.2.10 Consider documenting the token assumptions made by `safeApprove`

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: The `BlueBuyCallback.safeApprove` does not validate the token's existence on purpose assuming that such validation will be done by the `BlueBuyCallback.onBuy` caller.

Recommendation: Morpho should explicitly document this assumption like it has been already done in the bundler's TokenLib contract.

The strong assumption is that `Midnight.take` will perform that validation once `SafeTransferLib.safeTransferFrom` will be executed after the callback returns the flow to the caller contract.

Morpho: Fixed in PR 1093.

Spearbit: PR 1093 adds a documentation comment in `ERC20Lib` clarifying that token existence validation is delegated to the caller.

5.2.11 `createBlueBuyCallback()` reverts if an owner already has `BlueBuyCallback` deployed

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: In `BlueBuyCallbackFactory`, `createBlueBuyCallback()` always attempts to deploy a new `BlueBuyCallback` for the specified `owner` without checking if the contract already exists.

As such, if `BlueBuyCallback` is already deployed for an `owner`, calling `createBlueBuyCallback()` for the same `owner` will revert.

If a user batches `createBlueBuyCallback()` with other operations (eg. `setAuthorization()`, `BLUE.supply()`), an attacker can force the entire batch to revert by front-running and calling `createBlueBuyCallback()` first.

Recommendation: Consider checking if `BlueBuyCallback` was deployed first and perform a no-op if so:

```
address callback = callbackOf[owner];
if (callback != address(0)) return callback;
```

Morpho: Fixed in PR 1090.

Spearbit: Fix verified.

5.2.12 Offers in midnight using `BlueBuyCallback` can be forced to revert by draining Morpho Blue liquidity

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: In `BlueBuyCallback`, the `onBuy()` callback always withdraws `buyerAssets` from Morpho Blue:

```
if (buyerAssets > 0) IMorpho(BLUE).withdraw(marketParams, buyerAssets, 0, address(this),
↪  address(this));
```

This introduces a front-running attack vector, where an attacker could force a call to `take()` to revert by temporarily consuming liquidity in `BLUE` (eg. by borrowing enough such that `totalSupplyAssets - totalBorrowAssets < buyerAssets`).

Recommendation: Consider adding a comment that bundles/routers should call `buyerAssetsBound()` and `take()` in the same transaction to avoid this.

Morpho: Acknowledged.

Spearbit: Acknowledged.